

MLOps Lab Experiment 2 – EC-1

Containerize and Orchestrate a Model Deployment Pipeline (Docker + Airflow)

Student Name: Priyanshu Tripathi

Student ID: 2025ae05230

Deliverables included according to the EC-1 checklist supplied for Lab 2.

1. Part A – Docker Evidence

The Docker image was built and tagged as **cartvista-scorer:v1**. The submitted terminal evidence shows the image ID and size.

```
=> exporting to image 5.3s
=> => exporting layers 5.3s
=> => writing image sha256:ce6274759b0955a58e5041822c549345eb652ab93f0ced909ab0813290326a74 0.0s
=> => naming to docker.io/library/cartvista-scorer:v1 0.0s
(.venv) (base) s2025ae05230@ip-10-3-27-223:~/workspace/cartvista-mlops$ docker images cartvista-scorer
REPOSITORY TAG IMAGE ID CREATED SIZE
cartvista-scorer v1 ce6274759b09 20 seconds ago 879MB
(.venv) (base) s2025ae05230@ip-10-3-27-223:~/workspace/cartvista-mlops$
```

Figure 1. Docker build completion and docker images output showing cartvista-scorer:v1, image ID ce6274759b09, size 879MB.

Step A4 container-run evidence

The supplied screenshots do not contain the actual Step A4 docker run output. Therefore, this PDF does not fabricate an A4 result. A screenshot of the successful container run should be added before final submission.

2. Part B – Airflow: Fully Green Run

The Airflow evidence below shows successful execution of the CartVista retraining pipeline.

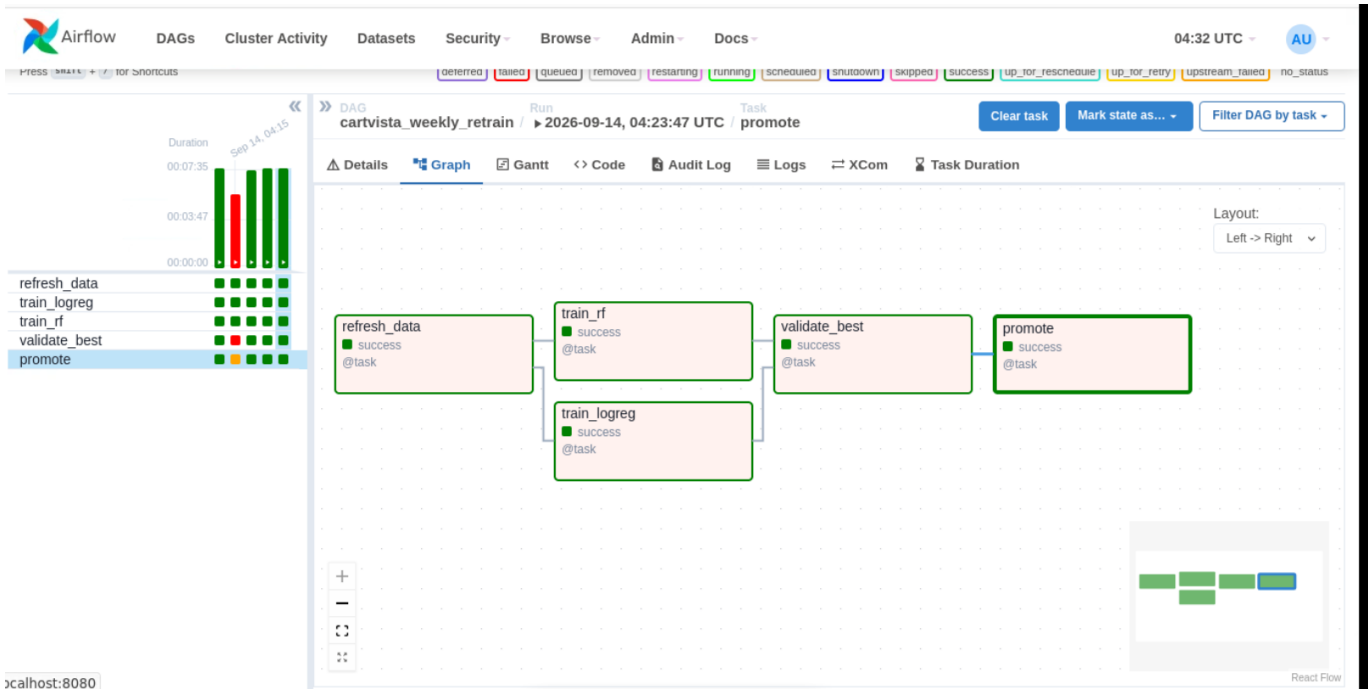


Figure 2. Airflow Graph view showing a fully successful pipeline run.

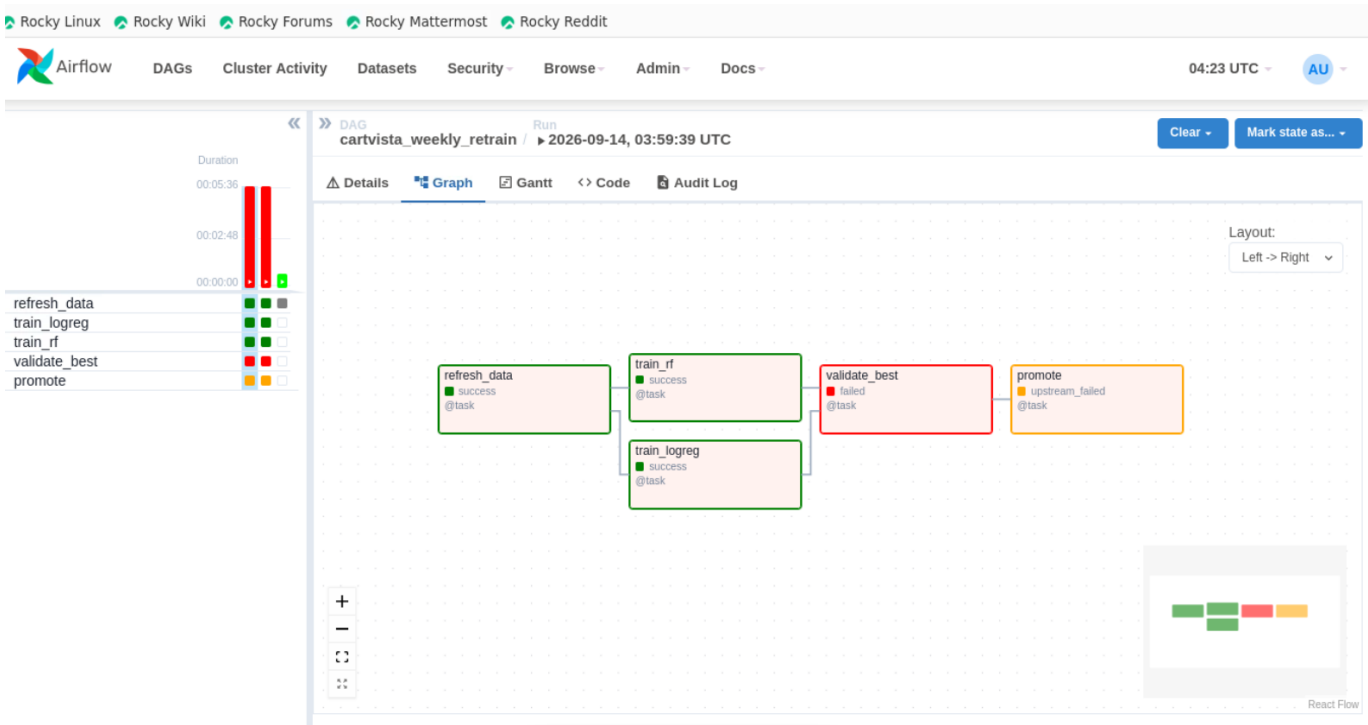


Figure 3. Airflow Grid/Graph evidence of successful tasks in the CartVista weekly retraining DAG.

3. Part B – Quality Gate Failure

The quality gate was deliberately changed to `AUC_GATE = 0.95`. The validation task failed because the best AUC was 0.7496, and promote became `upstream_failed`.

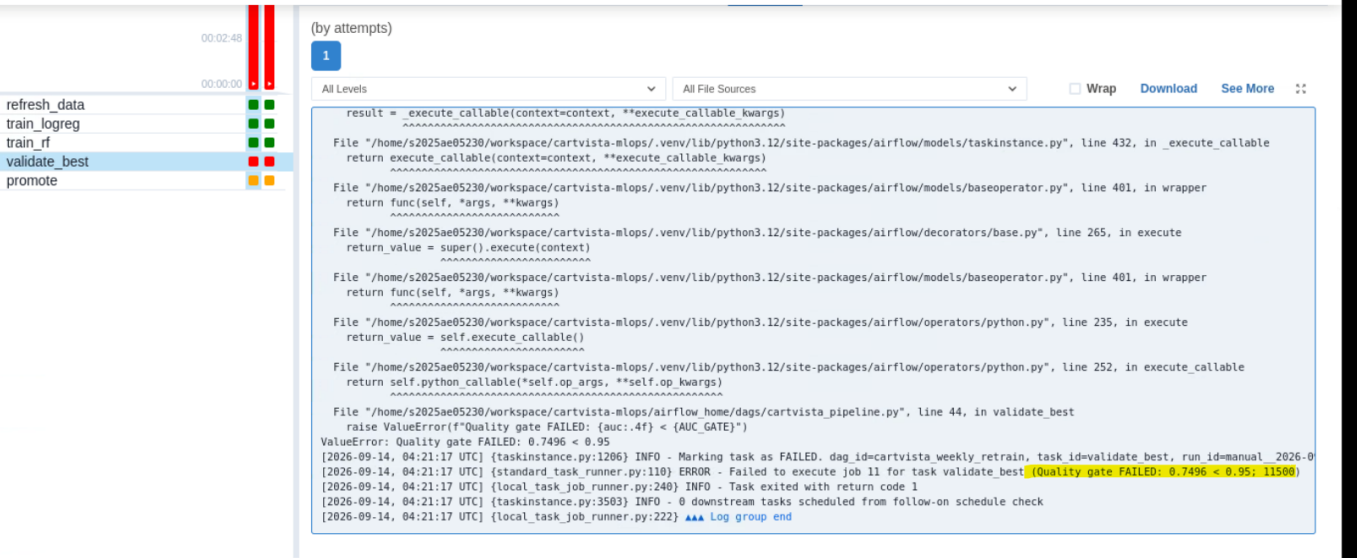


Figure 4. `validate_best` log showing: Quality gate FAILED: 0.7496 < 0.95.

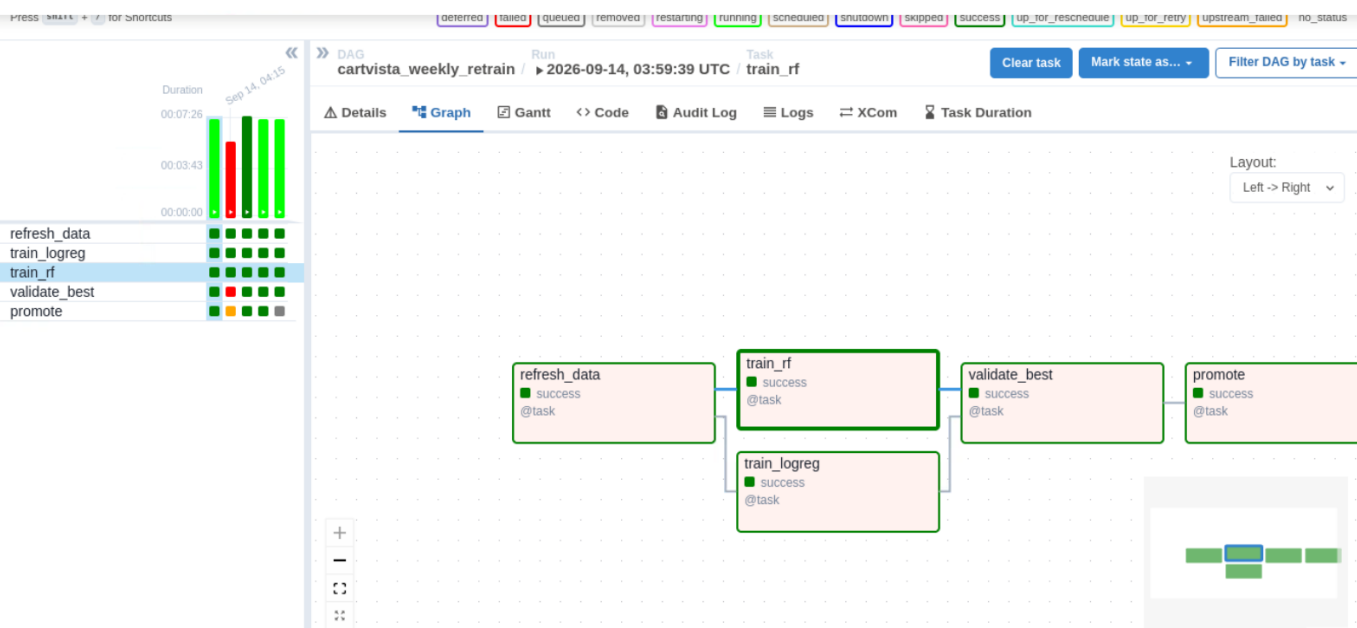


Figure 5. Airflow Graph view showing `validate_best` failed and `promote` marked `upstream_failed`.

4. E1 – Retry Policy and Idempotency

Retries are safe for the training tasks when the tasks are designed to be idempotent: rerunning them should produce the same intended final result without corrupting shared state. For a task that sends retention emails, retries can be dangerous because the same email could be sent more than once. Idempotency means that executing the same operation multiple times has the same intended final effect as executing it once.

5. E2 – Shrink the Docker Image

The submitted Docker evidence shows a current image size of **879MB** for cartvista-scorer:v1. The lab requires reducing the image below 450MB without breaking Step A4 and reporting both before and after sizes. A practical method is to keep only inference-time dependencies, remove packages that are only needed for training/development, rebuild with a clean cache, and verify that the scorer still loads the model and writes scored.csv.

Current status: Before = 879MB.

6. E3 – Schedule and catchup

The required schedule is every Monday at 02:00. The Airflow cron expression is **0 2 * * 1**. The DAG should use this expression instead of schedule=None. **catchup=False** is appropriate because this operational weekly pipeline should run from the current schedule onward rather than automatically creating missed historical runs when first enabled.

7. E4 – Containerized Task

E4 is optional. No completed score_weekly task execution or E4-specific DAG code is included in the supplied evidence, so it has not been claimed as completed.

8. Reflection

The gate stopped a bad model from being promoted, but it cannot stop a bad or misleading dataset from being used in the first place. If the input data is corrupted, biased, stale, incorrectly labeled, or otherwise unsuitable, the model can still pass a numerical quality gate based on that flawed data. This is why data validation and data-quality monitoring are important parts of a production ML pipeline.

9. Final Evidence Status

Deliverable	Evidence in this PDF
Step A4 container run + docker images	Docker image evidence included
Fully green Airflow run	Included
Red gate-failure run + promote upstream_failed	Included
E1 – Idempotency	Written answer included
E2 – Before/after size + method	Before size and method included
E3 – Cron + catchup reasoning	Written answer included
E4 DAG code	Not attempted / optional
Reflection	Included